Anexo de seguridad: arquitectura de control de acceso

Proyecto: GastoFlow, gestión de gastos corporativos (Ionic 9 + Angular 22 standalone + Supabase)
Módulo: Desarrollo de Interfaces (0488) · DAM 2 · Nebrija
Autor: Jorge González Gómez · **Versión:** v1.0.0

La seguridad se organiza en **tres capas independientes**. Si una falla, las otras siguen protegiendo los datos:

Capa	Dónde	Qué decide
1. Interfaz	menú, botones, widgets (`signals` computadas)	Qué ve el usuario
2. Router e interceptor	guards `canMatch` / `canDeactivate`, `authInterceptor`	A dónde puede ir y con qué credenciales viaja cada petición
3. Base de datos	políticas **RLS**, triggers y funciones de PostgreSQL (Supabase)	Qué filas puede leer o modificar, aunque alguien se salte la interfaz

A. Fundamentación teórico-técnica

A.1 Functional Guards (Angular / Ionic standalone)

Un **guard** es una función que el router ejecuta durante la navegación. Desde Angular 15 se escriben como **funciones** (`CanMatchFn`, `CanActivateFn`, `CanDeactivateFn`) en lugar de clases con `implements CanActivate`, que es la forma obsoleta. Obtienen sus dependencias con `inject()` y devuelven `true`, `false` o un `UrlTree`, que redirige; también pueden devolver una `Promise` o un `Observable` de cualquiera de ellos.

Ciclo dentro de una navegación:

1. Empieza la navegación (`NavigationStart`).
2. **Reconocimiento** de la URL, donde se ejecuta **`canMatch`**.
3. **Comprobación de guards**: `canDeactivate` de la página que se abandona y `canActivateChild` / `canActivate` de la nueva.
4. Resolvers.
5. Descarga del componente (`loadComponent`) y activación (`NavigationEnd`).

Tipo	Cuándo	Uso en GastoFlow
`canMatch`	Durante el reconocimiento, antes de aceptar la ruta	`authGuard`, `guestGuard`, `permisoGuard(...)`
`canActivate`	Con la ruta ya reconocida	No se usa: `canMatch` llega antes
`canDeactivate`	Al intentar salir de la página	`cambiosSinGuardarGuard` en el alta de gastos

`canActivate` frente a `canMatch`. - Con `canActivate`, el router primero da la ruta por buena. Si usa `loadChildren`, la configuración hija **ya se ha descargado** aunque luego se bloquee. - Con `canMatch`, si el guard falla la rama "no existe" para ese usuario: no se descarga ni se construye.

Además se quitó la precarga (`PreloadAllModules`) de la plantilla de Ionic. Con ella, el código de las páginas de administración se descargaría en segundo plano también para los empleados.

A.2 HTTP Interceptors (`HttpInterceptorFn`)

Cada petición de `HttpClient` atraviesa una **cadena de interceptores**: cada uno recibe la petición y la función `next` para pasarla al siguiente. Es el patrón *Chain of Responsibility*, como los `Filter` de los Servlets en Java.

- **Cabeceras.** `HttpRequest` es inmutable, así que se clona con `setHeaders`: `apikey` (la clave pública que exige Supabase) y `Authorization: Bearer <JWT>`. Solo se añaden a peticiones hacia la API REST propia, para no filtrar el token a terceros.
- **401 Unauthorized** (token caducado o inválido): se renueva la sesión **una sola vez** con el *refresh token* y se repite la petición. Para evitar bucles, la petición reintentada se marca con un `HttpContextToken`. Si la renovación falla, se cierra la sesión y se redirige a `/login?returnUrl=<página actual>`.
- **403 Forbidden** (con sesión pero sin permiso; por ejemplo, una política RLS lo impide): se avisa al usuario y **no** se cierra la sesión.
- **Renovación compartida.** Si varias peticiones reciben 401 a la vez, todas esperan a la **misma** promesa de renovación (`AuthService.refrescarSesion()`), y no se lanzan varias en paralelo.

A.3 RBAC (Role-Based Access Control)

RBAC (modelo NIST) relaciona **usuarios ↔ roles ↔ permisos**. Al usuario no se le dan permisos directamente: se le asigna un rol, y el rol agrupa permisos.

	Autenticación (AuthN)	Autorización (AuthZ)
Pregunta	¿Quién eres?	¿Qué puedes hacer?
En GastoFlow	Supabase Auth (email + contraseña → JWT), `authGuard`	`PERMISOS_POR_ROL`, `permisoGuard`, políticas RLS
Fallo HTTP	401	403

Encapsulamiento del token. - El JWT de Supabase lleva los claims `sub` (id del usuario), `email` y `exp` (caducidad). - El **rol no se toma del cliente**: `AuthService` lo lee de la tabla `perfiles`, que está protegida por RLS, y lo guarda junto a los claims en un único tipo, `SesionClaims`. - `AuthService` expone ese estado como signals de solo lectura: `sesion`, `rol`, `permisos`, `estaAutenticado`, más `tienePermiso()`. - Guards, interceptor, menú y páginas **preguntan al servicio**; ninguno lee el token ni el storage por su cuenta.

B. Implementación en el proyecto

B.1 Rutas, roles y servicios

Pantalla	Ruta	Guard / permiso	`empleado`	`admin`
Login, Registro	`/login`, `/registro`	`guestGuard` (solo sin sesión)	—	—
Inicio, Ayuda	`/inicio`, `/ayuda`	`authGuard`	Sí	Sí
Mis gastos	`/gastos`	`gastos:ver-propios`	Sí	Sí
Registrar gasto	`/gastos/nuevo`	`gastos:crear` + `cambiosSinGuardarGuard`	Sí	Sí
Aprobaciones	`/admin/aprobaciones`	`gastos:aprobar`	**Bloqueada**	Sí
Cuadro de mando	`/admin/dashboard`	`dashboard:ver`	**Bloqueada**	Sí

```
// src/app/core/models/rol.model.ts: la ÚNICA matriz de permisos del cliente
export const PERMISOS_POR_ROL = {
  empleado: ['gastos:ver-propios', 'gastos:crear'],
  admin: ['gastos:ver-propios', 'gastos:crear', 'gastos:ver-todos', 'gastos:aprobar',
          'dashboard:ver', 'informes:exportar', 'usuarios:gestionar'],
};

// src/app/app.routes.ts (extracto)
{ path: 'login', canMatch: [guestGuard], loadComponent: ... },
{ path: '', canMatch: [authGuard], loadComponent: () => import('./layout/shell/shell.component')
...,
  children: [
    { path: 'gastos/nuevo', canMatch: [permisoGuard('gastos:crear')],
      canDeactivate: [cambiosSinGuardarGuard], loadComponent: ... },
    { path: 'admin/aprobaciones', canMatch: [permisoGuard('gastos:aprobar')], loadComponent: ...
},
    { path: 'admin/dashboard',    canMatch: [permisoGuard('dashboard:ver')],  loadComponent: ...
},
  ] },
```

- **Guards asíncronos.** `authGuard` hace `await auth.listo` antes de decidir: espera a que Supabase recupere la sesión guardada, así un F5 en una página privada no expulsa al usuario. Si el token ha caducado, intenta renovarlo antes de mandar al login.
- **Redirección contextual:**
- Sin sesión → `/login?returnUrl=/admin/dashboard`; tras el login se vuelve a esa URL.
- Sin permiso → `/inicio?denegado=/admin/dashboard`, con un aviso accesible (`role="alert"`).
- **Registro del interceptor.** Se hace una sola vez, en `app.config.ts`, con `provideHttpClient(withFetch(), withInterceptors([authInterceptor]))`. Los datos SQL se piden con `HttpClient` a la API REST de Supabase (`/rest/v1/...`, en `ApiRestService`) precisamente para que todas las peticiones pasen por él. `supabase-js` se usa solo para Auth y Realtime.

B.2 Control visual condicional

- **Menú lateral.** Es un `computed()` que filtra las opciones por `auth.permisos()`. Un empleado no ve "Aprobaciones" ni "Cuadro de mando".
- **Botones de acción.** Aprobar y Rechazar se generan solo si `auth.tienePermiso('gastos:aprobar')`. Exportar CSV/PDF solo aparece con `informes:exportar`.
- **Sin "parpadeos".** Como el menú y los botones son signals, al cerrar sesión o cambiar de usuario la interfaz se recalcula sola.
- **Ocultar no es proteger.** La protección real está en la base de datos.

B.3 Seguridad en la base de datos (Supabase)

- **RLS activado en las 3 tablas:**
- Un empleado **solo lee y crea sus propios gastos** (`empleado_id = auth.uid()`).
- Solo puede editar o borrar los suyos mientras están **pendientes**.
- Administración (`es_admin()`) ve y revisa todos.
- **Trigger `gastos_reglas_negocio`:**
- La fecha no puede ser futura.
- Un empleado siempre crea en estado `pendiente`.
- Solo administración cambia el estado, y queda registrado quién y cuándo.
- **Registro.** Un trigger sobre `auth.users` crea el perfil con rol `empleado`: nadie puede registrarse como administrador desde la app.
- **Validación asíncrona del correo.** La función `email_disponible()` (`security definer`) devuelve solo `true`/`false` y no expone ningún dato.
- **Claves:**
- En el frontend solo va la clave **pública** (`anon`).
- La URL y la clave **no están en el repositorio**: se generan en `src/environments/supabase.config.ts`, que Git ignora, a partir de `.env` en local o de las variables de entorno de Vercel.
- La clave `service_role` nunca sale del panel de Supabase.
- **Cabeceras HTTP** en Vercel (`vercel.json`): `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (evita *clickjacking*), `Referrer-Policy` y `Permissions-Policy`.

C. Matriz de alternativas evaluadas

Guards funcionales vs. comprobación en el componente (`ngOnInit` / `ionViewWillEnter`)

Criterio	Guard en el router (elegido)	Comprobación en el componente
Seguridad	La página prohibida no se crea ni se pinta	Se crea y se ve un instante antes de echar al usuario; es fácil olvidarla en una pantalla
Red	Con `canMatch` no se descarga su código	El chunk se descarga siempre
Mantenibilidad	Toda la política está a la vista en `app.routes.ts`	Lógica repetida en cada componente
Escalabilidad	Una ruta nueva solo necesita añadir su guard	Cada pantalla nueva copia la comprobación

Interceptor HTTP vs. cabeceras manuales en cada servicio

Criterio	Interceptor (elegido)	Cabeceras manuales
Seguridad	Un único punto decide a qué dominio viaja el token	Riesgo de olvidar la cabecera o de enviarla a terceros
Errores 401/403	Centralizados, con renovación compartida	Cada servicio los trata a su manera
Mantenimiento	Cambiar el esquema de autenticación toca un archivo	Hay que tocar todas las llamadas
Sobrecarga	Una función más en la cadena (despreciable)	Código duplicado

RBAC vs. ABAC vs. flag `isAdmin`

Criterio	`isAdmin`	RBAC (elegido)	ABAC
Expresividad	Dos estados	Roles con conjuntos de permisos	Reglas sobre atributos del usuario, del recurso y del contexto
Escalar a un rol nuevo	Hay que tocar todo el código	Una línea en `PERMISOS_POR_ROL`	Escribir reglas nuevas
Auditoría	`if` repartidos por el código	Una única matriz legible	Necesita un motor de políticas
Encaje en GastoFlow	Insuficiente	**Adecuado** (2 roles, 7 permisos)	Excesivo. La única regla "por atributo" (un empleado solo ve **sus** gastos) la resuelve RLS en la base de datos

Conclusión. GastoFlow combina **guards funcionales con `canMatch`** (sin precarga), un **interceptor HTTP único** con renovación de token y **RBAC con permisos atómicos** centralizado en `AuthService`. Todo ello está respaldado por **RLS y triggers** en la base de datos, que son la barrera final.